Backpropagation — Exam Question 1 (with answers)

1.1 A scalar sigmoid unit. Consider the scalar model $f = \sigma(Wx + b)$.

(a) Decompose the forward pass into its primitive operations. (2 pts)

(b) Using the variables from (a), draw the computation graph. (2 pts)

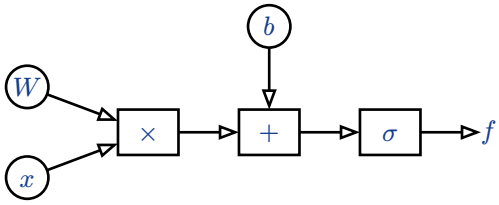
(c) For $x = 1$, $W = 2$, $b = 3$, target $y = 1$, and squared-error loss $L = (f - y)^2$, compute the forward pass and the gradient at each node of the graph. You may annotate the graph or write forward and backward values for each operation. (6 pts)

(d) Derive $\partial f / \partial W$ analytically. You may use $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. (1 pt)

Answer. ***(a)***

$$
\begin{aligned}
z_1 &= Wx \quad && \text{(multiply)} \\
z_2 &= z_1 + b && \text{(add)} \\
f &= \sigma(z_2) && \text{(sigmoid)}
\end{aligned}
$$

(b)



(c) Forward.

$z_1 = Wx = 2, \quad z_2 = z_1 + b = 5, \quad f = \sigma(5) \approx 0.9933, \quad L \approx 4.48 \times 10^{-5}.$

Backward.

$$
\begin{aligned}
\frac{\partial L}{\partial f} &= 2(f - y) \approx -1.34 \times 10^{-2} \\
\frac{\partial L}{\partial z_2} &= 2(f - y) f(1 - f) \approx -8.90 \times 10^{-5} \\
\frac{\partial L}{\partial z_1} &= \frac{\partial L}{\partial b} = \frac{\partial L}{\partial z_2} \approx -8.90 \times 10^{-5} \\
\frac{\partial L}{\partial W} &= \frac{\partial L}{\partial z_1} \cdot x \approx -8.90 \times 10^{-5}, \quad \frac{\partial L}{\partial x} = \frac{\partial L}{\partial z_1} \cdot W \approx -1.78 \times 10^{-4}
\end{aligned}
$$

(d)

$$\frac{\partial f}{\partial W} = \sigma'(Wx + b) \cdot x = \sigma(Wx + b)(1 - \sigma(Wx + b)) \cdot x.$$

At $x = 1$, $b = 0$ this is $\sigma(W)(1 - \sigma(W))$.

1.2 The sigmoid-derivative curve. Fig. 1 plots the $\partial f / \partial W$ formula you derived in 1.1(d), evaluated at $x = 1$, $b = 0$, as a function of W.

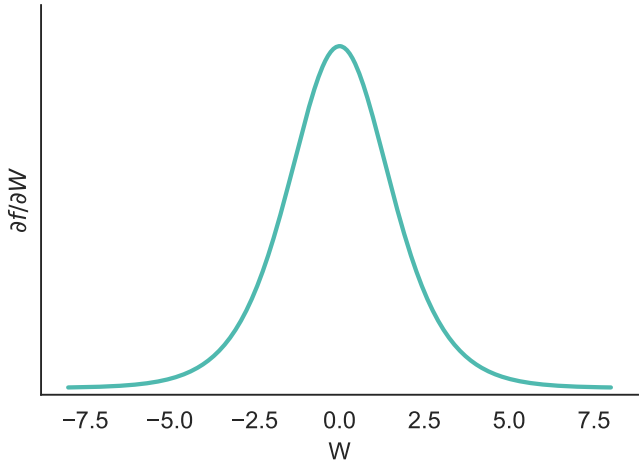


Fig. 1

(a) Explain the shape of the curve in Fig. 1. (1 pt)

(b) Using the chain rule, derive $\partial L/\partial W$ for the scalar model under $L = (f - y)^2$. (1 pt)

(c) Explain why $\partial L/\partial W = 0$ when $f = y$. (1 pt)

(d) State the gradient descent update rule for W. Why would large initial values of W lead to poor loss minimization? (2 pts)

Answer. ***(a)*** *Bell, symmetric about $W = 0$, peak $\frac{1}{4}$ at $W = 0$; decays to 0 in the tails as σ saturates and one of σ, $(1 - \sigma)$ vanishes.*

(b)

$$\frac{\partial L}{\partial W} = 2(f - y) \cdot \sigma'(Wx + b) \cdot x.$$

(c) *$f - y = 0 \Rightarrow \partial L/\partial W = 0$. A perfect fit leaves nothing to minimize.*

(d) *$W \leftarrow W - \eta \frac{\partial L}{\partial W}$. Large $|W|$ pushes z deep into the sigmoid tails where $\sigma'(z) \approx 0$, so the update vanishes — the "dead gradient" regime, in which W barely moves per step.*

1.3 Two-layer scalar composition. Now consider the two-layer composition

$$G = f_2(f_1(x; W_1); W_2) = \sigma(W_2\sigma(W_1x + b_1) + b_2)$$

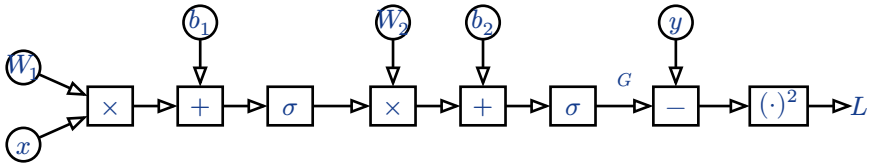
and the same squared-error loss $L = (G - y)^2$.

(a) Draw the computation graph for the full model and loss. (1 pt)

(b) Compute the forward pass and the gradient at each node, for scalar values $x = 1$, $W_1 = 2$, $b_1 = 1$, $W_2 = 4$, $b_2 = 1$, target $y = 1$. (4 pts)

(c) Explain why the gradient of W_1 is much smaller than the gradient of W_2. (1 pt)

Answer. ***(a)***



(b) *With $h = \sigma(W_1x + b_1)$, $z_2 = W_2h + b_2$:*

Forward.

$$z_1 = 3, \quad h \approx 0.9526, \quad z_2 \approx 4.810, \quad G \approx 0.9919, \quad L \approx 6.53 \times 10^{-5}.$$

Backward.

$$\frac{\partial L}{\partial G} = 2(G - y) \approx -1.62 \times 10^{-2}$$
$$\frac{\partial L}{\partial z_2} = 2(G - y)G(1 - G) \approx -1.30 \times 10^{-4}$$
$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z_2} \cdot h \approx -1.23 \times 10^{-4}, \quad \frac{\partial L}{\partial b_2} \approx -1.30 \times 10^{-4}$$
$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial z_2} \cdot W_2 \approx -5.18 \times 10^{-4}$$
$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \cdot h(1 - h) \approx -2.34 \times 10^{-5}$$
$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \cdot x \approx -2.34 \times 10^{-5}, \quad \frac{\partial L}{\partial b_1} \approx -2.34 \times 10^{-5}$$

(c) *The chain to W_1 passes through one more sigmoid than the chain to W_2, picking up the first-sigmoid derivative $h(1 - h) \approx 0.045$ together with an upstream $W_2 = 4$ factor; the h at the tail of the W_2 chain cancels in the ratio, giving $\partial L/\partial W_1 \approx W_2(1 - h) \cdot \partial L/\partial W_2 \approx 0.19 \cdot \partial L/\partial W_2$ — the $\sim 5\times$ gap. With more layers, this saturation factor compounds.*

1.4 Fully-connected variant and weight initialization. Consider a fully-connected variant of the two-layer model from 1.3:

$$h = \sigma(\boldsymbol{W}_1 x + b_1), \quad G = \sigma(\boldsymbol{W}_2 h + b_2),$$

where the input $x \in \mathbb{R}^2$ is now a 2-feature vector, $\boldsymbol{W}_1 \in \mathbb{R}^{4\times 2}$ and $\boldsymbol{W}_2 \in \mathbb{R}^{1\times 4}$ are weight matrices (boldface to distinguish them from the scalar W_1, W_2 of 1.3), $b_1 \in \mathbb{R}^4$, $b_2 \in \mathbb{R}$, and $h \in \mathbb{R}^4$ is the hidden activation. The scalar chain from 1.3 corresponds to a single *stream* of this network — one entry of $\boldsymbol{W}_1$ times one entry of $\boldsymbol{W}_2$, shown as the highlighted path in Fig. 2 — which reduces exactly to the scalar 1.3 model only when every off-path entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ is zero.

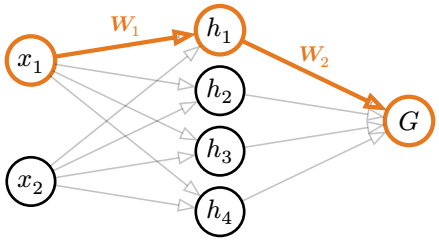


Fig. 2

We train this network on the two-moons binary classification dataset (Fig. 3). The four loss curves (A–D) in Fig. 4 correspond to four different weight initializations ($\eta = 0.01$ for 5000 epochs): A — all zeros; B — Gaussian ($W_{ij} \sim \mathcal{N}(0, 1)$); C — large magnitude ($W_{ij} \sim \mathcal{N}(0, 10^2)$); D — symmetric ($W_{ij} = 1$ for every entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$).

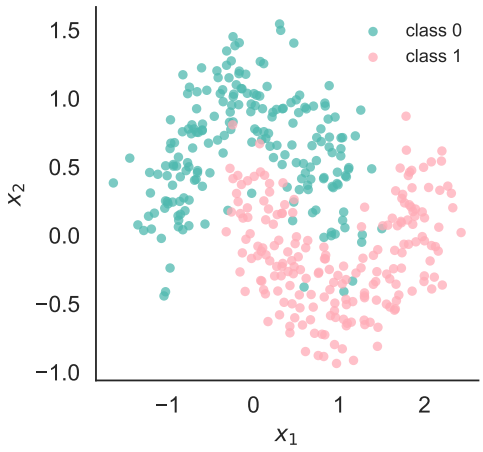


Fig. 3

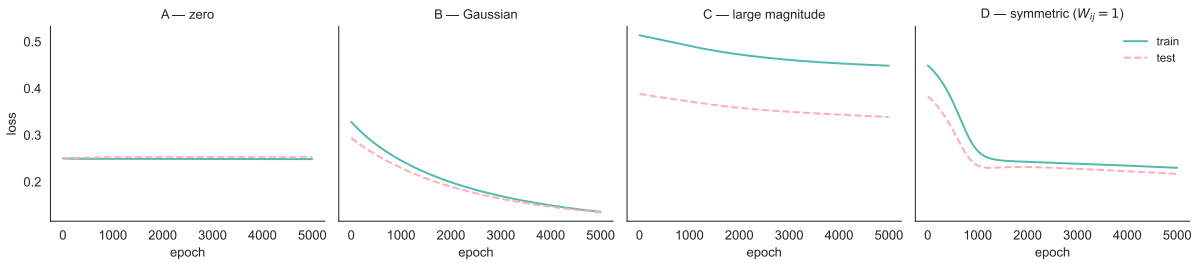


Fig. 4

(a) Decompose the forward pass into its constituent operations, annotating the shape of each intermediate at every step. (2 pts)

(b) In one sentence per plot, explain the behavior of each of the four curves, tying back to your analyses in 1.2 and 1.3. (4 pts)

(c) The dataset is a binary classification task. In addition to the squared-error loss we have used so far, name one other loss that would be appropriate here and briefly justify the choice. (1 pt)

Answer. ***(a)***

$$\begin{aligned} z_1 &= \boldsymbol{W}_1 x + b_1 \in \mathbb{R}^4 \\ h &= \sigma(z_1) \qquad \in \mathbb{R}^4 \\ z_2 &= \boldsymbol{W}_2 h + b_2 \in \mathbb{R} \\ G &= \sigma(z_2) \qquad \in \mathbb{R} \end{aligned}$$

(b)

- A — zero. *Flat at* 0.25*: all weights* 0*, so first-layer gradients carry a* $W_2 = 0$ *factor (1.2b) and every hidden unit is identical; only* b_2 *learns, giving constant* $G \approx 0.5$*.*
- B — Gaussian. *Healthy.* $|z|$ *near* 0 *keeps* $\sigma'(z)$ *near its max* $\frac{1}{4}$ *(1.2a); the 1.3 stream propagates gradient cleanly; units break symmetry.*
- C — large. *Stuck.* $|z|$ *enormous so* $\sigma'(z) \approx 0$ *(1.2a); the stream in 1.3 multiplies two such vanishing factors; only* b_2 *drifts.*
- D — symmetric ($W = 1$). *Descends, then plateaus above B. Gradients flow, but every hidden unit sees identical* $z_j = x_1 + x_2$ *and the same gradient — the four units update in lockstep, so the net has the capacity of one unit.*

(c) Binary cross-entropy: $L_{\text{BCE}} = -[y \log f + (1-y)\log(1-f)]$*. Two reasons: (i) MLE for Bernoulli targets, penalizing confident wrong predictions more steeply; (ii) with sigmoid output,* $\partial L_{\text{BCE}}/\partial z = f - y$ *— the output* σ' *factor cancels, removing the saturation slowdown MSE suffers.*

1.5 LayerNorm. Consider an extension of the model in 1.4 where a *LayerNorm* is inserted after the first linear layer and before the hidden sigmoid:

$$z_1 = \boldsymbol{W}_1 x + b_1, \quad \hat{z}_1 = \frac{z_1 - \mu}{s}, \quad h = \sigma(\hat{z}_1),$$

where $h \in \mathbb{R}^4$ is the hidden activation passed to the output layer (same role as in 1.4), and μ, s are the per-sample mean and standard deviation of z_1 across the four hidden units:

$$\mu = \frac{1}{4}\sum_{i=1}^{4} (z_1)_i, \quad s = \sqrt{\frac{1}{4}\sum_{i=1}^{4} \big((z_1)_i - \mu\big)^2}.$$

Fig. 5 shows training loss on the moons data under the same poor (large-magnitude) initialization as 1.4(C), both with and without LN (left panel). The right panel shows the magnitude of the gradient at every entry of $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$, captured at epoch 1000 (dashed grey line in the left panel), when the performance of the non-LN model is only slowly improving.

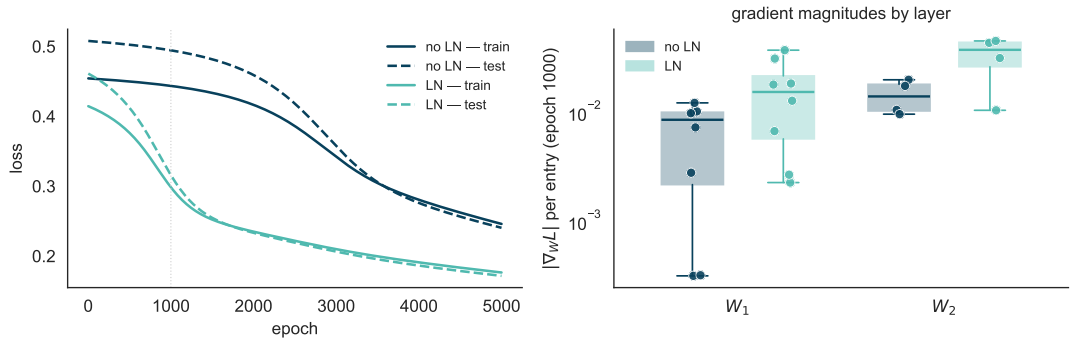


Fig. 5

(a) Redraw the computation graph for a single stream of this modified network. (2 pts)

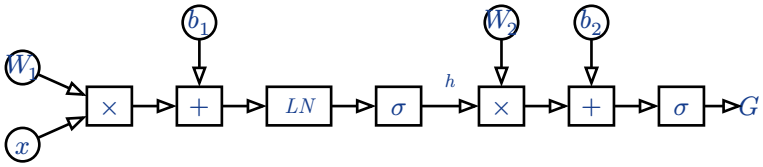
(b) What is the added term to the chain of gradients flowing through the network? (1 pt)

(c) Using the $\partial f/\partial W$ curve in Fig. 1, explain how LayerNorm shifts the pre-activation z_1 toward the region where σ' is large, and why this makes the LN variant more *robust* to different initialization scales of $\boldsymbol{W}_1$. (3 pts)

(d) Using the right panel of Fig. 5, explain why the LayerNorm variant converges much faster than the no-LN model. (1 pt)

(e) Using the $\partial f/\partial W$ curve in Fig. 1, explain why the gradient magnitude is larger at both $\boldsymbol{W}_1$ and $\boldsymbol{W}_2$ under LayerNorm. (1 pt)

Answer. ***(a)*** *Single stream, with LN inserted between the first add and the first sigmoid:*



(b) *For a single stream, treating μ and s as fixed scalars, $\hat{z}_1 = (z_1 - \mu)/s$ gives*

$$\frac{\partial \hat{z}_1}{\partial z_1} = \frac{1}{s}.$$

The chain therefore picks up an extra factor of $\frac{1}{s}$ — LN rescales backward gradients by the reciprocal of the pre-activation standard deviation. (The full Jacobian across all four units additionally couples them through μ and s, but the single-stream factor is simply $\frac{1}{s}$.)

(c) *Fig. 1 shows σ' peaked at $\frac{1}{4}$ when the argument is 0 and decaying to 0 in the tails. LN pulls z_1 to mean 0 and unit variance across the hidden units, so every pre-activation sits within $\sim \pm 2$ of the peak — regardless of $|\boldsymbol{W}_1|$.*

Without LN, $|z_1|$ scales with $|\boldsymbol{W}_1|$: small init lands near the peak and training works; large init lands in the dead tails and training stalls — so the trajectory is sensitive to the init scale. With LN, $\hat{z}_1$ has mean 0 and unit variance regardless of $|\boldsymbol{W}_1|$, and $\sigma'(\hat{z}_1)$ is decoupled from the init scale. This does not make LN strictly *better (a well-tuned init matches it, see 1.4B), but it removes init-scale as something the practitioner must get right.*

(d) *Right panel: LN gradients sit $\sim 2\times$ above no-LN at both layers. Under $W \leftarrow W - \eta \nabla_W L$, a larger gradient produces a larger per-step change; over 5000 epochs LN accumulates many more useful updates and the loss descends accordingly.*

(e) *From (c), LN keeps $\sigma'(\hat{z}_1) \approx \frac{1}{4}$; without LN, $\sigma'(z_1) \approx 0$. The σ' factor enters the chain* directly *for $\boldsymbol{W}_1$ and* indirectly *for $\boldsymbol{W}_2$ — saturated h pushes z_2 into saturation too, whereas healthy h under LN keeps z_2 off the tails. Saturation therefore propagates to both layers.*

1.6 K-nearest neighbors. A *K-nearest-neighbors* (KNN) classifier assigns to each input the majority label among its K closest points in the training set (by Euclidean distance). Unlike the sigmoid MLP from 1.4, KNN has no trainable parameters — it simply stores the training data and computes distances at prediction time.

Fig. 6 shows the decision boundaries of four classifiers on the moons training data: the MLP from 1.4(B) (Gaussian init, trained to convergence) and three KNN classifiers. Which is which has been anonymized as "Model A"–"Model D". The training set contains $N_0 = 106$ class-0 points and $N_1 = 94$ class-1 points ($N = N_0 + N_1 = 200$).

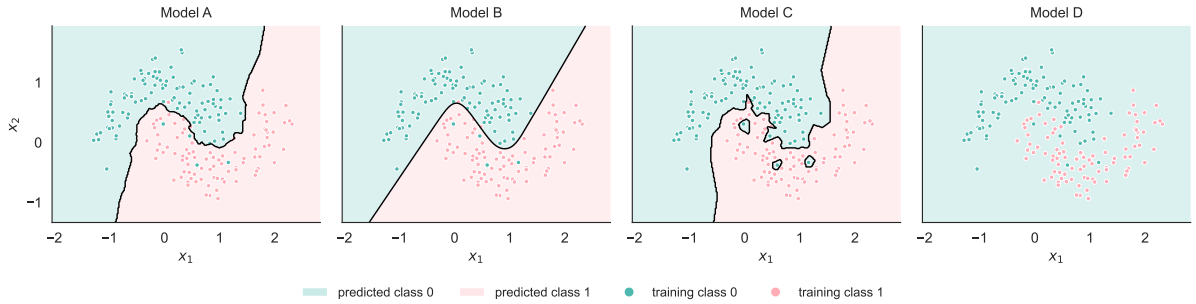


Fig. 6

(a) Name one practical advantage and one practical disadvantage of KNN relative to the trained MLP. (2 pts)

(b) The NumPy skeleton below implements a KNN classifier for binary labels. Fill in the three blanks (marked `...`) so that the function returns the predicted label for each row of `X_test`. (3 pts)

```
import numpy as np

def knn_predict(X_train, y_train, X_test, K):
    """
    X_train: (N, d) training inputs
    y_train: (N,)   training labels in {0, 1}
    X_test:  (M, d) query points
    Returns: (M,) predicted labels in {0, 1}
    """
    preds = np.zeros(X_test.shape[0])
    for i, x in enumerate(X_test):
        # 1. Euclidean distances from x to every training point: shape (N,)
        dists = ...
        # 2. indices of the K nearest training points
        idx   = ...
        # 3. majority label among those K neighbors
        preds[i] = ...
    return preds
```

(c) What is the computational cost of a single prediction at test time, as a function of the training-set size N and the input dimension d? How does this compare with a forward pass through the 2-4-1 MLP from 1.4? (1 pt)

(d) The sigmoid MLP was trained by gradient descent. Why can a KNN classifier *not* be trained the same way? (1 pt)

(e) Three of the four panels (A, B, C) show KNN classifiers with $K \in \{1, 5\}$ together with the MLP. Match each of Models A, B, C to one of {MLP, KNN with $K = 1$, KNN with $K = 5$}, and justify your assignments from the shape of the boundaries. (2 pts)

(f) Panel D shows a KNN classifier with $K = N = 200$. Explain why its prediction is constant across the plane, referring to the class balance stated above. (1 pt)

Answer. ***(a)*** Advantages. *Zero training cost — just store the data, no optimization or hyperparameters beyond K; and no parametric assumption, so the class of boundaries KNN can express grows with the dataset.* Disadvantages. *Prediction is $O(Nd)$ per query, so inference scales with the training-set size; the classifier is sensitive to label noise, where a single mislabelled point near the boundary directly flips predictions in its neighbourhood; and performance degrades in high dimensions where Euclidean distances become uninformative.*

(b)

```
dists    = np.linalg.norm(X_train - x, axis=1)
idx      = np.argsort(dists)[:K]
preds[i] = (y_train[idx].mean() > 0.5).astype(float)
```

Line 1 broadcasts x against the (N, d) training matrix and takes the row-wise Euclidean norm to get an $(N,)$ distance vector. Line 2 sorts the distances and keeps the K smallest indices. Line 3 averages the neighbor labels (in $\{0, 1\}$ the mean is the fraction of class-1 votes) and thresholds at $1/2$, equivalent to the majority rule $\hat{y} = \mathbb{1}\left[\sum_{i \in \mathcal{N}_K} y_i > K/2\right]$.

(c) *$O(Nd)$ per query: $O(d)$ to compute each of the N training-point distances, then $O(N)$ to find the K smallest. The 2-4-1 MLP is $O(dH + H) = O(dH)$ with $H = 4$ hidden units — independent of N. KNN prediction therefore scales with training-set size; the MLP's does not.*

(d) *KNN has no trainable parameters — the "model" is the training data itself. Any continuous handle enters through an* $\arg\min$ *inside the nearest-neighbor lookup, which is piecewise-constant, so the gradient of the loss with respect to it is zero almost everywhere. Gradient descent therefore has nothing useful to minimize over.*

(e) *A = KNN $K = 5$; B = MLP; C = KNN $K = 1$.*

- C (K=1): *most jagged, each training point owns a small Voronoi cell; label noise shows up as tiny outlier islands.*
- A (K=5): *piecewise-linear with moderate pockets — local voting smooths the $K = 1$ cells but still can't produce a continuous curve.*
- B (MLP): *single smooth curve — the 0.5 level set of $\sigma(\boldsymbol{W}_2\sigma(\boldsymbol{W}_1 x + b_1) + b_2)$ is necessarily continuous and is sculpted by only four hidden sigmoids.*

(f) *With $K = N$ every query sees the entire training set, so the neighbor average equals the global mean label $N_1/N = \frac{94}{200} = 0.47 < 0.5$ at every point. The majority vote therefore returns class 0 everywhere — KNN at $K = N$ reduces to the constant majority-class predictor.*